

WATERLOO ROCKETRY

Git Tutorial

Software Sub-System

Qian Qian · Shrey Shingala



WATERLOO ROCKETRY

Schedule

Today's route through Git

01

Git Background

Origins and mental model

02

Git Clients

CLI, lazygit, and GUI clients

03

Remote Setup & GitHub

GitHub and remotes

04

Git Setup & Basics

Setup, stage, commit, inspect

05

Team Collaboration

Branches, merges, PRs, reviews

06

Hands-On

Practice the full workflow



Good SWE Practices

Habits that help a team ship reliable software

01

Clear Naming

Clear commits and branches

02

Write Clearly

Clear PR descriptions

03

Small Changes

Focused pull requests

04

Code Review

A second set of eyes

05

Detailed Testing

Verify before merging

06

Document Things

Decisions, usage, and comments



0 Announcements

Announcements

1. Do your safety training!

(We might do a short pause during this tutorial for people having to attend the bay safety tour to go do that and come back.)

2. We will start a thread in the team Slack or create a form to collect your GitHub username. This is needed to send you an invitation to the Waterloo Rocketry GitHub organization.



1 Git Background

Why Git Exists

Git lets a team make changes independently without losing shared history.

Versioned History

Every commit records a checkpoint: who changed what, when, and why. You can compare, restore, and audit work without copying folders.

Distributed By Design

Every clone gets you a full copy of the repository with all the history. Work offline, make changes and additions to the history, then synchronize.



Git History Is A Graph

Commits link to parents: Git stores a graph, not copies of entire projects.



Commit = snapshot · Parent = earlier commit · Branch = movable name



The Git Mental Model

Most Git confusion disappears when you separate these four places.

Working Tree

The files on your computer. Edit here.

Staging Area

The exact snapshot selected for the next commit.

Local Repository

Your committed history and branches on this machine.

Remote Repository

The shared copy on GitHub, usually named `origin`.



A Commit Is A...

Use the model that helps with the question in front of you.

Snapshot (Actually)

Git represents a commit as a directory tree plus metadata and parent pointers. Checking out an old commit does not replay every prior diff.

Diff

Best for review: what changed between two states? GitHub, Tower, and lazygit make this view immediate.

Further reading: Julia Evans, "Do we think of git commits as diffs, snapshots, and/or histories?"



2 Git Clients

Git Clients

CLI

Best for learning the model, scripting repeatable work, and seeing every operation explicitly.

TUI

`lazygit` is a useful terminal interface for staging, browsing history, and resolving routine work quickly.

GUI

Tower and GitHub Desktop make history, diffs, and repository state easy to inspect visually.

Choose Deliberately

Use the client that makes you confident. Learn the CLI vocabulary anyway: it transfers across tools and is invaluable when troubleshooting.



Set Up Your Client

Configure your identity and authentication once in the client you will use day to day.

GitHub Desktop

Sign in with your GitHub account and confirm the organization is visible. Its credential flow handles cloning and pushing.

Tower Or Another Client

Set your Git identity and preferred authentication method in Settings (differ from client to client). Use a personal access token for HTTPS or an SSH key for SSH.



3 Remote Setup & GitHub

Create Your GitHub Account

Your GitHub account is your identity for repositories, reviews, organization access, and contribution history.

Register

Open `github.com/signup`, choose a professional username, and use an email address you can access. Complete the verification email before continuing.

Secure It

Enable two-factor authentication, save recovery codes somewhere safe, and add a profile name people on the team can recognize.



Create Your GitHub Account (ii)

For Waterloo Rocketry

We will start a thread in the team Slack or create a form to collect your GitHub username.

We will then send an invitation to the Waterloo Rocketry GitHub organization.

Accept the organization invitation sent to your GitHub account. Confirm that the organization appears in the account menu before trying to push to a team repository.



Create The Empty GitHub Repository

You already have a local Git repository. Create its matching home on GitHub without adding a second initial commit.

1. New Repository

On GitHub, select + then New repository. Choose your personal account or the organization that should own the project.

2. Name & Visibility

Use the intended project name. Choose Private for team work unless the project is deliberately public.

3. Leave It Empty

Do not initialize with a README, .gitignore, or license. Your existing local repository already has its own history.



Copy The Remote Address

After creating the repository, copy its URL from the Quick Setup page or the Code button. Pick one connection method and use it consistently.

HTTPS

Works everywhere. Authenticate with GitHub CLI or a credential manager; GitHub does not accept account passwords for command-line Git operations.

SSH

Convenient once configured. Generate an SSH key for this computer, add its public key to GitHub, then copy the

`git@github.com:OWNER/REPOSITORY.git` URL.



Attach The Remote To Your Existing Repository

In a terminal, open the existing project folder. `origin` is the conventional name for its primary GitHub remote.

Add

```
git remote add origin git@github.com:OWNER/REPOSITORY.git
```

Verify

```
git remote -v
```



Attach The Remote To Your Existing Repository (ii)

Read The Result

The verification output should show the same `origin` URL for fetch and push. Check the owner and repository name carefully before you publish anything.



Publish The Existing History

Push the branch you already have. The `-u` flag remembers the matching GitHub branch, so later `git push` and `git pull` know where to go.

Check The Branch

```
git branch --show-current
```

First Push

```
git push -u origin main
```



Publish The Existing History (ii)

If Your Branch Is Not `main`

Replace `main` with the output from `git branch --show-current`. Refresh the GitHub repository page after the push and confirm the files and commit history arrived.



Inspect Or Repair A Remote

Address

Check that `origin` points to the expected GitHub organization and repository before you push.

Branches

Fetch to learn which branches exist on GitHub and whether your branch is ahead or behind its remote-tracking branch.

Permission

A successful fetch confirms read access. Push only to the branch and repository your team expects.

Already Have `origin`?

Do not add it again. Inspect with `git remote -v`, then correct an outdated address with `git remote set-url origin <REMOTE-URL>`.



4 Git Setup & Basics

Configure Your Git Identity

Set the name and email Git writes into every commit. Use the email address verified on GitHub so your work appears on your profile.

Your Name

```
git config --global user.name "Your Name"
```

Your Email

```
git config --global user.email "you@example.com"
```



Configure Your Git Identity (ii)

Why `--global`?

It sets the default for every repository on this computer. A project can override it with the same command without `--global` when needed.



Start A New Project

Use this path when you are beginning with a folder of files and neither a local repository nor a GitHub repository exists yet.

1. Create The Folder

```
mkdir flight-dashboard  
cd flight-dashboard
```

2. Initialize Git

```
git init
```

3. Confirm

```
git status
```



Start A New Project (ii)

What `git init` Does

It creates the hidden `.git` directory that stores commits, branches, and configuration. Your project files remain ordinary files until you add and commit them.



Create The First Commit

Create the initial files in your editor first: a README, source folder, project configuration, and `.gitignore` are common starting points. Then save the first snapshot.

Inspect

```
git status
```

Stage

```
git add .
```

Commit

```
git commit -m "chore: initialize repository"
```



Create The First Commit (ii)

Before ``git add .``

Check `git status` and your `.gitignore`. Never add credentials, API keys, build outputs, or local editor settings that do not belong in shared history.



Publish The New Project On GitHub

On GitHub, create a New repository with the intended owner, name, and visibility. Leave its README, `.gitignore`, and license unchecked because your local project already has its first commit.

Connect GitHub

```
git remote add origin git@github.com:OWNER/  
REPOSITORY.git
```

Publish

```
git push -u origin main
```



Publish The New Project On GitHub (ii)

Done

Refresh the GitHub page to confirm the files and `Initial Commit` appear. Your local folder and GitHub repository are now connected; future work follows `status` → `add` → `commit` → `push`.



Get A Repository And Check Its State

Clone starts a new local copy with its history and remote already configured. Initialize when files already exist on your computer. `git status` is the first command to run whenever you are unsure what Git sees.

Clone

```
git clone https://github.com/OWNER/REPOSITORY.git
```

Inspect

```
git status
```



Choose The Right Start

Use `git clone` when you are starting with a remote repository. Use `git init` when you are starting with a local folder of files, with no existing Git history.

After you have a local repository, the general workflow will be the same for all your work.



The Everyday Loop

1. Inspect

Pull the latest work, read the issue, and check your branch status.

2. Change

Make one focused change. Run the relevant formatter, tests, and build.

3. Stage

Review the diff, then stage only the intended files.

4. Commit

Write a concise Conventional Commit that explains the change.

5. Share

Push the branch and open a pull request with context and evidence.

6. Learn

Respond to review, update the branch, and capture lessons in documentation.



Make A Focused Commit

Review first, stage only the change you intend to save, then create one descriptive checkpoint.

Review

```
git diff
```

Stage

```
git add path/to/file
```

Commit

```
git commit -m "feat: add  
sensor validation"
```



Stage Only What You Intend

Use `git add path/to/file` for an intended file. `git add .` stages every change below the current folder, so inspect `git status` again before committing.



Sync And Read History

Bring in the team's changes before sharing yours, then inspect the compact history when you need context.

Get Changes from Others

```
git pull
```

or

```
git fetch && git merge origin/main
```

Publish Your Work

```
git push
```

History

```
git log
```



Sync And Read History (ii)

When `pull` Stops

Git may need you to resolve a conflict if the same lines changed in two places. Read the message, resolve deliberately, and run the relevant checks before continuing.

Difference Between `pull` and `fetch`

`git pull` is a shortcut for `git fetch` followed by `git merge`. Use `fetch` when you want to inspect the incoming changes before merging them into your branch. You can inspect unmerged incoming changes with `git log origin/main` or `git diff origin/main`.



Fix Mistakes Safely

Before Sharing

Use the Changes view to discard a mistaken hunk or unstage a file. Check the diff first: discarding is destructive to uncommitted work.

After Sharing

Use the history view to create a revert commit through the client or a new PR. Leave the correction visible and reviewable.

Shared History Needs Care

Do not rewrite a branch that other people are building on. For a shared branch, prefer `git revert` so the correction is explicit and recoverable. Ask before force-pushing.



5 Team Collaboration

Branches Are Cheap

A branch is a movable name pointing at a line of commits. Create one per focused piece of work.

Update Main

In your client, fetch or pull the current `main` branch before starting new work.

Create A Branch

Use New Branch and choose a short descriptive name such as `fix/default-route` or `feature/dashboard-autosave`.

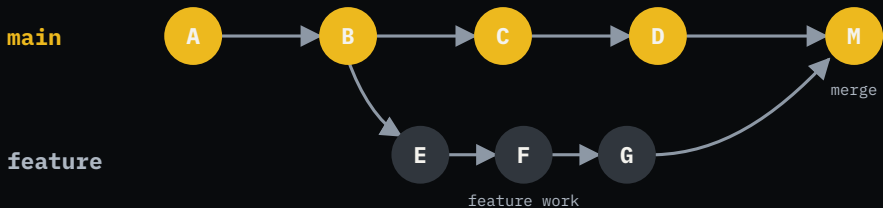
Publish It

Push or publish the branch when you are ready to request feedback. The client will associate it with the remote.



Branches Visualized

Feature work branches from `main` and merges back when it is ready.



Keep one branch focused on one question: what change are we making, and how will we know it works?



Create, Switch, And Publish A Branch

Start feature work from the current `main` branch. A descriptive branch tells teammates what the work is for before they open a diff.

Create

Create and move to a new branch:

```
git switch -c feature/sensor-fix
```

Switch Existing Branches

Move between branches you already created:

```
git switch main  
git switch feature/sensor-fix
```



Create, Switch, And Publish A Branch (ii)

List Branches

```
git branch -a
```

Publish

```
git push -u origin feature/sensor-fix
```

Before You Switch

Run `git status` first. Commit, stash, or discard work you do not want to carry over; Git will stop a switch that would overwrite your changes. Use `git switch -` to return to the branch you were on immediately before.



Return And Merge When Allowed

Return To Main

```
git switch main
```

Update It

```
git pull
```

Merge

```
git merge feature/sensor-fix
```

Clean Up

After a merged branch is no longer needed, delete its local name with `git branch -d feature/sensor-fix`. This does not delete the commits that were merged into `main`.



Return And Merge When Allowed (ii)

However, at Waterloo Rocketry, we use ***pull requests*** (PRs) to request review and merge work. A PR is a conversation about a branch of commits, not a single commit. After we review a PR related to the branch, we merge it into `main` and delete the branch. So you mostly will never run `git merge` directly; GitHub does it for you when we merge the PR through the web interface, and you only need to pull the result back to your local repository's `main` branch.



Collaboration Workflow

Plan

Read the issue, confirm scope, and identify acceptance criteria before editing code.

Build

Use a focused branch. Keep commits small and run relevant checks locally.

Review

Open a PR/MR with context, evidence, risks, and a link to the issue.

Discuss

Treat review comments as collaboration. Ask questions, explain trade-offs, and resolve every thread.

Merge

Merge only when required checks and approvals are complete. Follow the repository's merge policy.

Close The Loop

Link the PR/MR to the issue, verify the result, and update documentation or release notes.



Merge Conflicts

Update Your Branch

Use Fetch and Update Branch in a GUI, or the branch-sync action in lazygit, before opening or merging a PR.

Resolve The Intent

Use the client's conflict editor to read both changes. Build the correct final state; do not blindly choose ours or theirs.

Verify Again

Conflicts can change behaviour. Re-run the relevant tests and inspect the final diff before marking the conflict resolved.



6 Good SWE Practices

Clear Naming

Commit Messages

Use Conventional Commits: `type: description`, or `type(scope): description`. For example:

- `feat: add dashboard autosave`,
- `fix: correct default route`, or
- `test: cover LabJack parsing`.

Common types are `feat`, `fix`, `docs`, `test`, `refactor`, and `chore`.

Branch Names

Use `<member>/<convention>/<description>`:

- `watiam/fix/default-route`,
- `watiam/feature/dashboard-autosave`, or
- `watiam/docs/user-guide`.

Include an issue number when that helps tracking.



Small Changes

One Idea Per Change

When a commit or PR does two unrelated things, split it. Small units are easier to review, test, revert, and explain.

Separate The Noise

Keep refactors, generated files, formatting sweeps, and unrelated cleanup out of a functional change whenever possible.

Make Review Easy

A small PR gives reviewers enough context to reason about behaviour and enough time to check it carefully.



Write Clearly

Problem

What user or system need does this solve? Link the issue.

Approach

What changed, and why was this design chosen?

Evidence

Which tests, screenshots, logs, or manual checks demonstrate the result?

Call Out Risk

State migrations, compatibility concerns, follow-up work, or checks you could not run. Clear uncertainty is more useful than hidden uncertainty.



Detailed Testing

Choose The Right Level

Unit tests protect small logic; integration tests verify components together; manual checks validate a user-facing flow. Use the smallest set that gives useful confidence.

Test The Edges

Include failure paths, empty states, invalid inputs, permissions, and recovery behaviour. Bugs often live at the boundary, not the happy path.

Record What Ran

In the PR, state the checks you ran and their result. If a check could not run, say why and describe the remaining risk.



Document Things

For Users

Update setup steps, configuration, behaviour, and limitations whenever the interface changes.

For Maintainers

Document decisions, invariants, integration points, and non-obvious trade-offs where future contributors will need them.

Comments Explain Why

Use comments for constraints and rationale that code cannot express. Do not narrate line-by-line mechanics that the code already makes clear.



7 Hands-On

8 Wrap-Up

Before You Push

Scope

One goal, no accidental generated files, and no secrets.

History

Clear branch name, focused commits, and an up-to-date base branch.

Evidence

Formatter, tests, build, screenshots, logs, or a stated limitation.

Final Check

Open your client's status and compare views, then read the PR as if you were reviewing it for the first time.



Client Action Cheat Sheet

GitHub Desktop

Changes for diff and staging; History for commits; Branch for creation and publication; Repository for pull and push.

Tower

Working Copy for status; History for graph and diff; Branches for creation and merge; Conflict editor for resolution.

lazygit

Files for staging; Commits for history; Branches for switching; Remotes for synchronization. Press `?` to discover keys.



Closing

Make history that your teammates can understand.

The Habit

Inspect first. Make one focused change. Test it. Commit clearly. Ask for review. Leave the repository easier to work in than you found it.

Questions?

